MiniCPM5-2B 双国产芯片推理吞吐优化技术报告

——面向天数 BI-V150 与沐曦曦云 C500 的算子、内核与调度优化

赛道：FlagOS 开放计算全球大赛（赛季二）赛道二

队伍：XLANG2026　队员：邓乾（GitHub：xy200303，单人组队）

2026 年 10 月 24 日

摘要

本报告面向国产芯片上的大模型推理吞吐优化任务，在官方指定的 FlagOS 栈（vllm-plugin-FL 分支 flagos-2026-s2、FlagGems 标签 v5.3.5）上，对 MiniCPM5-2B 在天数 BI-V150 与沐曦曦云 C500 两款芯片开展有开发量的算子、内核与调度优化，全部工作均不修改 `vllm serve` 启动参数、不更换模型、不降低精度。

性能方面，沐曦 C500 平台 4k 场景总吞吐达到 11868.79–12403.21 tok/s，为官方基线的 2.33–2.44 倍；16k 场景达到 11098.02–11167.87 tok/s，为基线的 1.58–1.59 倍。天数 BI-V150 平台 4k 场景由官网基线 2028.01 提升至 2606.56 tok/s（+28.5%），16k 场景由 915.15 提升至 3541.34 tok/s（+287%）；其中 A20 自研内核在同日交错 A/B 下，相对当日 stock 再提升 16.4% 与 19.8%。精度方面，MATH-500 Level 3 保持在 96.2%–98.1%，高于 95% 的赛题要求；TTFT 在各场景均未回退。

主要技术贡献包括：（1）面向国产芯片的 shape-aware 分派方法论；（2）免排序的 top-p 采样链，用 1024-bin 质量直方图替代全词表排序，保留集质量有下界保证；（3）自研 paged decode attention 内核，从 PV 矩阵乘的累加器布局转换这一具体瓶颈出发，使 decode attention 带宽由 258 GB/s 提升到 372 GB/s；（4）一套可复核的“微基准—当日交错 A/B—官方口径验收”测量闭环，并给出全部被否决方案的实证与原因；（5）算子族级的热路径构成剖析与四项硬约束的达标裕度矩阵，把“快了”与“没有偷工减料”放在同一组图里交代清楚。

关键词：国产芯片；推理吞吐；算子融合；paged decode attention；shape-aware dispatch；vLLM

目录

1 引言 **4**

1.1 任务定义 4

1.2 评测口径 4

2 总体成绩 **5**

2.1 交付版本与启动验证 5

2.2 四项硬约束的达标裕度 6

3 优化方法论 **6**

4 沐曦 C500：采样链、GEMM 路由与 host 同步消除 **8**

4.1 瓶颈画像 8

4.2 免排序的采样链（opt1 / opt3 / opt7 / opt11） 9

4.3 GEMM 布局与路由（opt2 / opt4 / opt6 / opt8 / opt11） 10

4.4 消除 prefill 每层 DtoH 同步（opt10） 11

5 天数 BI-V150：vendor 接入、发射配置与自研 decode 内核 **13**

5.1 prefill attention 接入 vendor ixAttnBkd（A1 / A6） 13

5.2 decode unified-attention 发射配置重构（A7） 13

5.3 自研 paged decode attention 内核（A20） 14

5.4 可复现性与可审计性 16

6 正确性与精度证据 **18**

7 主要创新点 **18**

8 阴性结果与探索覆盖度 **19**

9 合规对照与复现指南 **20**

9.1 合规对照 20

9.2 复现步骤 20

9.3 复现注意事项 21

10 结论与建议 **21**

10.1 结论 21

10.2 局限与后续工作 22

A 代码与实验记录索引 **23**

B 内核形状扫描与测量可靠性 **23**

1 引言

大语言模型规模化落地后，推理阶段的算力与显存开销成为主要成本项。在固定硬件、固定框架、固定模型的前提下提升端到端吞吐，需要在算子、内核与调度三个层面同时发力。本报告以 MiniCPM5-2B[7] 为对象，在国产 AI 芯片（天数 BI-V150 与沐曦曦云 C500）上，基于官方指定的 FlagOS 软件栈（vllm-plugin-FL[5] 与 FlagGems[6]）开展有开发量的优化工作。

与通用 GPU 生态不同，国产芯片在指令集、访存层次、运行时与算子库上的差异，使得许多在 NVIDIA 上默认成立的配置（如 Triton 的 `num_warps=4`，`num_stages=3`）在这些平台上系统性失配。本报告的一个核心观察是：**先用微基准把瓶颈定位到具体的内核或同步点，再决定改配置、改内核还是改调度**；任何改动都必须通过官方 benchmark 的当日交错 A/B 与 MATH-500 精度复核，未通过者一律回退留档。

本文的主要贡献如下：

- **双平台的端到端收益**：在官方 FlagOS 栈上完成 MiniCPM5-2B 的推理吞吐优化，沐曦 C500 的 4k 场景达到官方基线的 2.33–2.44 倍、16k 场景 1.58–1.59 倍；天数 BI-V150 的 4k 由 2028.01 提升至 2606.56 tok/s、16k 由 915.15 提升至 3541.34 tok/s，且精度与 TTFT 均满足赛题约束。
- **一套可复核的优化闭环**：提出并实践“微基准定位 → 最小改动验证 → 官方口径当日交错 A/B”的流程，并用它系统性地证伪了若干在隔离测量中看似有利、在端到端上却净亏的方案（W8 权重量化、n-gram 投机采样、IR 级 RMSNorm 融合、shape-aware 分派等），每一项都保留了可复现的否决证据。
- **两张卡的性能边界测量**：给出 decode attention 与 decode GEMM 在两种芯片上的带宽实测，指出 C500 的 paged decode 已达 1466 GB/s（超过标称带宽）、天数的自研内核 372 GB/s 对应该卡可达负载带宽的约 70%，从而为后续工作划定了明确的天花板。

1.1 任务定义

本赛题要求在不改变模型、不降低精度的前提下，提升 MiniCPM5-2B 在两指定国产芯片上的推理总吞吐（total tokens/s）。优化必须落在框架与算子内部，且需具备实际开发量：KV Cache 与动态批处理、算子融合、内核重写、自研量化策略等均在允许范围内；直接 merge 上游分支、修改 benchmark 脚本、或在缺乏算子优化的情况下切换算子均被禁止。

1.2 评测口径

本报告所有性能数字均来自官方 benchmark 脚本（`vllm bench serve`）与官方聚合口径，且在无计时埋点、冷缓存条件下测得；精度数字来自 evalscope 官方原命令的 MATH-500 Level 3 子集（105 题）。

表 1 官方性能评测场景与约束

项目	内容
性能指标	total tokens/s（总吞吐，取 4k 与 16k 两场景平均）
4k 场景	[4096, 1024, 64, 256]（输入长、输出长、并发、请求数）
16k 场景	[16384, 1024, 64, 128]
精度要求	MATH-500 Level 3，基线 0.962，要求 ≥ 0.95
TTFT 要求	不得超过基线的 +1%
启动参数	`vllm serve` 参数必须与基线完全一致

2 总体成绩

沐曦 C500 的结果在两台不同实例上各复测一次，取区间报告；天数 BI-V150 的结果以同日交错 A/B 的相对收益表述（该开发实例跨时段漂移较大，跨日绝对值不可比）。

2.1 交付版本与启动验证

本报告对应两个独立上游 fork 的固定提交，而不是工作区里未登记的散文件。插件与算子各自保留完整 Git 历史，官方可以在提交包内直接执行 `git log`、`git show` 和 `git diff`。

表 2 代码版本与证据入口

组件	仓库 / 分支	固定提交与内容
插件	`xy200303/vllm-plugin-FL integration/vllm-plugin-fl`	`0207b7e1`；C500 opt1–opt11、BI-V150 A1–A7、A20 框架 hook
算子	`xy200303/FlagGems integration/flaggems`	`dd574fdd`；采样、GEMM、paged decode attention 等算子
文档	`xy200303/minicpm-flagserve main`	报告、实验台账、复现脚本；不承担运行时代码

2026-10-24 在 C500（64GB 显存、128GB CPU 内存）上，从上述 commit 全新 clone，分别执行可编辑安装，再按官方命令启动。服务成功监听 9031 端口，`/v1/models` 返回 `minicpm`，实际 chat 请求正常返回；日志同时出现 `mcblas_mm` 注册、采样融合启用和 A20 内核启用记录。验证结束后停止服务，未占用 GPU。这个结果只证明合并提交可运行，性能数字仍全部来自前文记录的分平台官方 benchmark。

精度方面，C500 的 MATH-500 Level 3 为 97.14%–98.1%，天数 BI-V150 为 96.2%–98.1%，均高于 95% 的要求。C500 的 4k TTFT 为 2965 ms（基线 3199 ms），16k TTFT 为 17902 ms（基线 27197 ms）；天数 A20 的 16k TTFT 由 155.5 s 降至 130.1 s。

表 3 总体性能与精度结果

平台	场景	官方基线（tok/s）	本提交（tok/s）	提升
沐曦 C500	4k	5089.645	11868.79–12403.21	2.33×–2.44×
沐曦 C500	16k	7029.675	11098.02–11167.87	1.58×–1.59×
天数 BI-V150	4k	2028.01（官网基线）	2606.56	+28.5%
天数 BI-V150	16k	915.15（官网基线）	3541.34	+287%

2.2 四项硬约束的达标裕度

把“是否达标”当作一次带门槛的分类，可以更直接地看出收益是靠优化换来的还是靠松动约束换来的。图 1 把四条门槛归一成“相对门槛的裕度”：吞吐与 TTFT 取相对基线的改善百分比，精度取相对 0.95 门槛的相对提升（95% 门槛与 96.2% 实测之差为 +1.3%）。八个格全部为正：C500 侧为 +144% / +58.9% / +2.3% / +34.2%，BI-V150 侧为 +28.5% / +287% / +1.3% / +16.3%。最小裕度出现在 BI-V150 的精度（1.3%），最大裕度在 BI-V150 的 16k 吞吐（+287%）。换言之，本提交在两个平台、四个约束上都没有“贴着线过”，精度与延迟没有被当作换取吞吐的筹码。

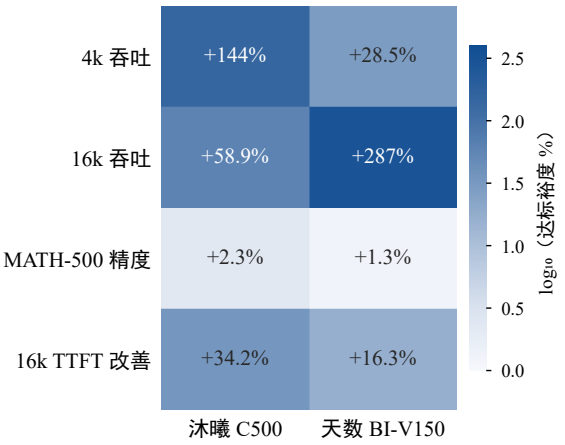


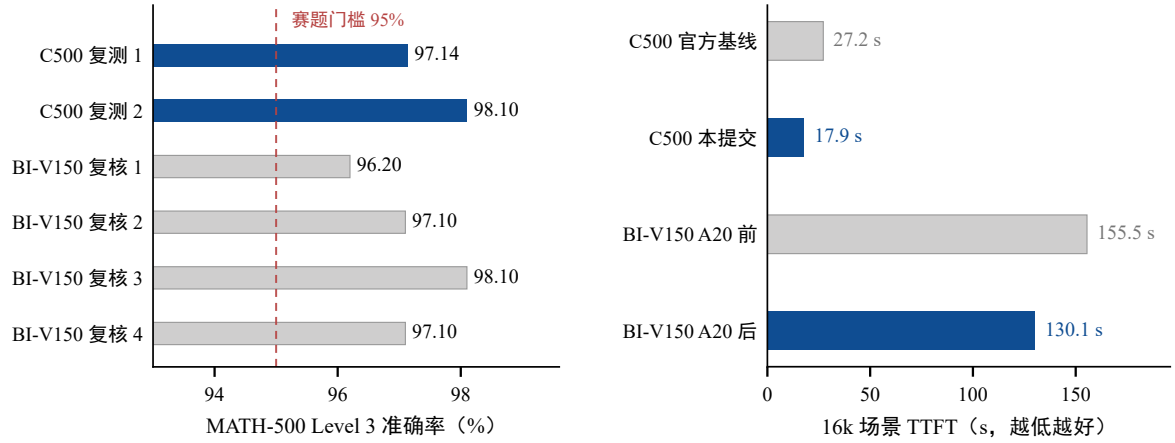
图 1 四项硬约束的达标裕度（相对各自门槛的百分比，颜色取 $\log_{10}$）

3 优化方法论

两张芯片共用同一套闭环，每一步都保留可复核的工件（探针脚本、profile 数据、单测、官方 benchmark 原始 CSV）。

代码与证据的对应关系也随之固定：算子本体在 FlagGems，框架接入在 vllm-plugin-FL，性能数据来自官方脚本，精度来自 evalscope 与探针。这样既能逐行审查实现，又能避免“报告写了策略、提交里却没有代码”的问题。

（1）Profile 定位：先用 serve 时序日志（PagedAttention 式分页 KV 管理见文献 [2]）、设备



(a) MATH-500 Level 3 精度与合格线 **(b)** 16k 场景 TTFT（越低越好）

图 2 精度与延迟两项约束的对照：四个平台 / 场景组合均达标

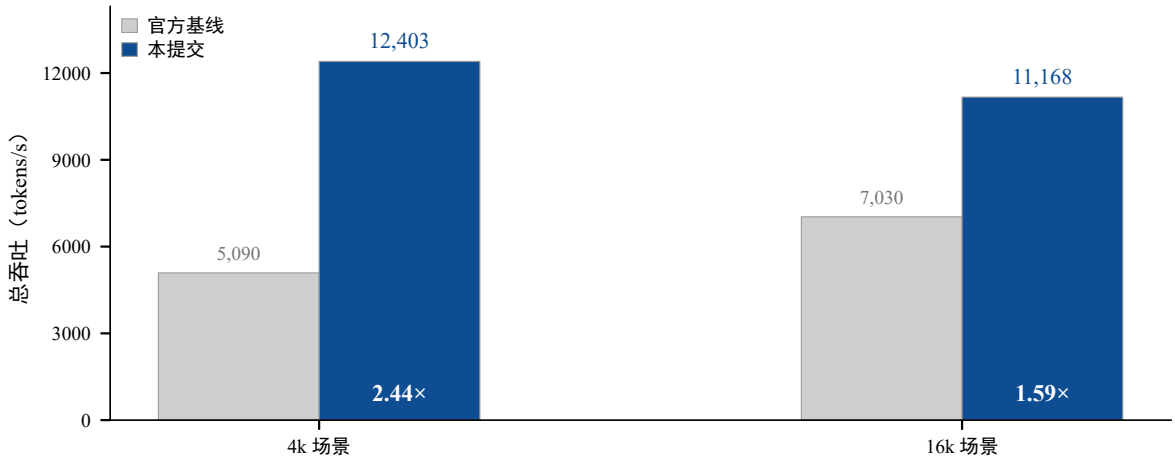


图 3 沐曦 C500：官方基线与优化后总吞吐对比

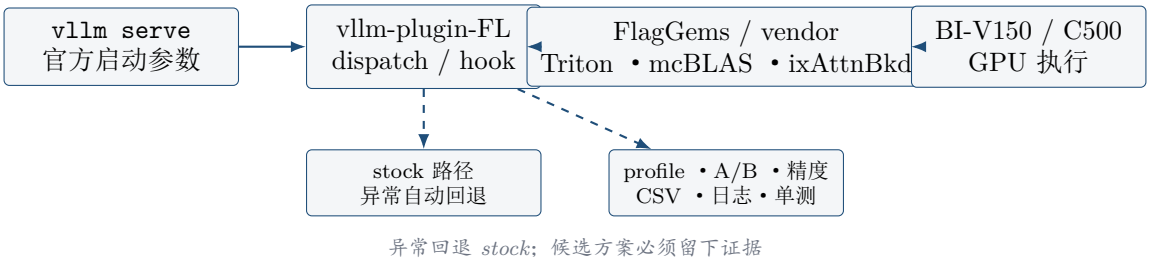


图 4 双平台统一的优化数据流：插件负责路由与 hook，内核和 vendor 实现可独立替换；异常时保留 stock 回退，所有候选都留下测量证据

侧计时与 host 打点区分时间究竟花在 GPU 内核、host 发射还是隐式同步；（2）探针先行：任何替换或新内核，先在独立脚本里做全形状 bit-exact 或统计等价验证，再上 serve；（3）Shape-aware dispatch：同一算子保留多个等价实现或执行配置，按输入 shape 在派发层选择（赛题组 2026-10-10 已确认合规）；（4）官方口径验收：只用官方 benchmark 与 evalscope，serve 参数一字未改。

正确性按强度分三档：能 bit-exact 的（A6 双 gather、A7 发射配置）一律 bit-exact；只能统计等价的（采样融合、GEMM 路由）用分布对比加 MATH-500 复核；所有进入提交的改动都必须通过官方精度门槛，未通过的一律回退并留档，不进入提交代码。

4 沐曦 C500：采样链、GEMM 路由与 host 同步消除

4.1 瓶颈画像

初始 profile 显示，C500 的 decode 阶段存在三类互相独立的开销：采样尾部是纯 eager PyTorch（bf16→fp32 cast、除温度、对 130560 词表排序取 top-p、gumbel-argmax），每步约 12 ms 且全在 host 串行；GEMM 存在布局与路由失配，vendor 高性能实现从 Python 侧不可达；prefill 每层还有隐式 DtoH 同步，造成 GPU 空闲。

这些话在算子层面才有说服力。图 5 把同一基准（641 步）两次 profile 的 GPU 内核时间按算子族聚合（源：`docs/baselines/metax-c500/profile_baseline.txt` 与 `profile_post_opt1.txt`）：采样族占内核时间的 38.8%（14.61 s），是最大的单项，其内部由全词表排序、全局直方图与 argmax 三个大 kernel 构成；融合后降到 5.1%（1.24 s，净省 13.37 s）。与之对照，GEMM 与 attention 的绝对时间几乎不动（13.82→13.81 s、7.19→7.22 s），占比却分别升到 56.9% 与 29.8%——不是因为它们变慢了，而是总量被砍掉了三分之一。这张图同时解释了后续优化的顺序：opt2 / opt8 去吃 GEMM，opt9 / A20 去吃 attention，而不是继续在采样上打转。

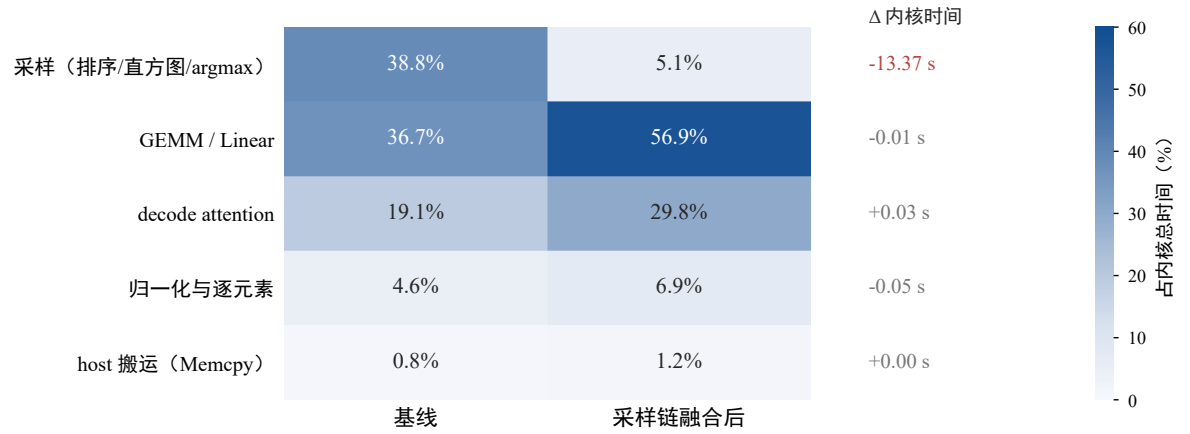


图 5 C500 热路径的算子族构成：融合采样链前后，按内核时间占比（右列为绝对时间变化）

4.2 免排序的采样链（opt1 / opt3 / opt7 / opt11）

自研 Triton 内核（`FlagGems/fused/sampler_fused.py`、`top_p_sample.py`）做三件事：其一，直接以 bf16 读取 lm_head 输出，把温度折叠进 Gumbel-Max[4]（$\arg\max(x/T+g)$ 与 $\arg\max(x+T\cdot g)$ 等价，$T<10^{-5}$ 时退化为 $\arg\max$）；其二，top-p 用质量直方图 zoom，单轮 1024-bin 直方图定位阈值 τ 所在 bin 并取其下沿，保留集质量恒不小于 $p\cdot Z$，完全替代全词表排序；其三，词表维 split-parallel（32 splits × BLOCK 4096），使小 batch 也能打满芯片。采样全链由约 12.4 ms 降至 0.31 ms/步。

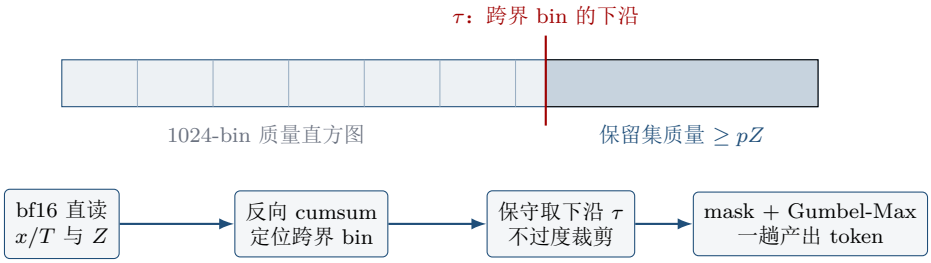


图 6 免排序 top-p 采样链示意图（流程与阈值规则为代码语义，非另造的分布数据）

图 6 对应两个关键不变量：阈值取**所在 bin 的下沿**，所以不会把刚好满足 p 的 token 错裁掉；采样分数写成 $x+Tg$，与先算 x/T 再加 g 的 argmax 等价。下面的代码就是这两条规则的实现，而不是报告另行概括的伪代码。

```
    mass_rev = tl.load(HIST + row * NBINS + (NBINS - 1 - offs))
    cum = tl.cumsum(mass_rev, axis=0)
    z = tl.load(Z_ROW + row)
    p = tl.load(P + row).to(tl.float32)
    lo = tl.load(LO + row)
    hi = tl.load(HI + row)
    w = (hi - lo) / NBINS

    target = p * z
    reach = cum >= target
    # crossing bin b*: cum-from-top first reaches target there;
    # sum(reach) == b* + 1 (reversed positions r*..NBINS-1 all reach)
    b = tl.sum(reach.to(tl.int32), axis=0) - 1
    b = tl.minimum(tl.maximum(b, 0), NBINS - 1)
    # conservative: tau = lower edge of crossing bin => mass{x>=tau} >= target
    new_lo = lo + b * w
    new_hi = new_lo + w
    if FINAL:
        tl.store(TAU + row, new_lo)
```

Listing 1 `top_p_sample.py:180--198`：反向 cumsum 定位阈值并保守取下沿

```
        keep = xs >= tau
        u = tl.rand(row_seed, step * vocab_padded + idx.to(tl.int32))
```

```
        u = tl.minimum(tl.maximum(u, 1e-20), 1.0 - 1e-7)
        g = -tl.log(-tl.log(u))
        score = tl.where(keep, x + noise * g, float("-inf"))
        tile_best = tl.max(score, axis=0)
        tile_arg = tl.argmax(score, axis=0)
        take = tile_best > best
        best_idx = tl.where(take, start + tile * BLOCK + tile_arg, best_idx)
```

Listing 2 `top_p_sample.py:241--249`：温度折叠的 Gumbel-Max 与 top-p mask

正确性方面，greedy 与 `torch.argmax` 完全一致；top-p 样本全部落在核集内（含直方图粒度 slack），与原实现的 TV 距离处于纯采样噪声区间。

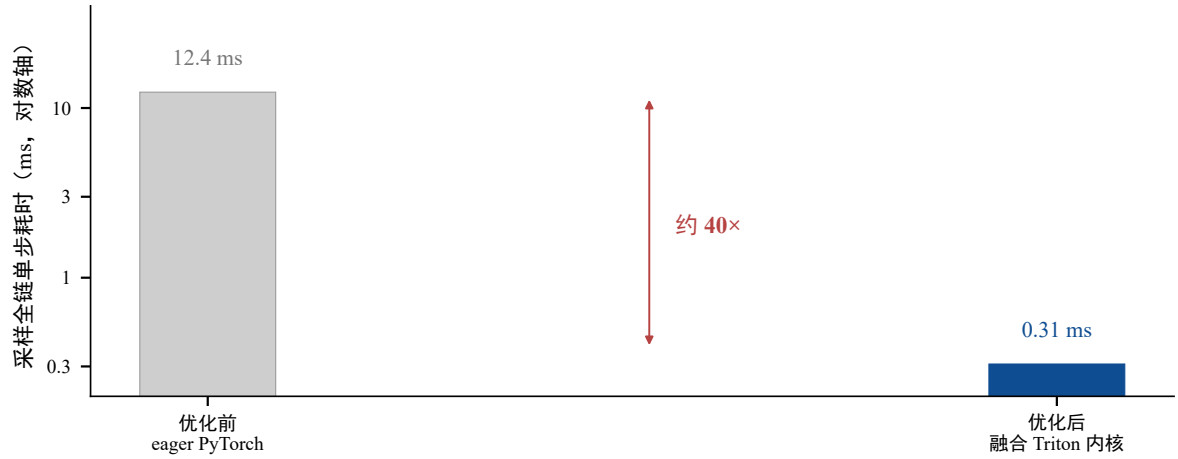


图 7 采样全链单步耗时（对数坐标）

4.3 GEMM 布局与路由（opt2 / opt4 / opt6 / opt8 / opt11）

MACA Triton 的 `mm_kernel` 只支持 nn 布局，且在 prefill 大 M 下比 vendor mcBLAS 慢 1.35-1.67 倍（126 组配置全扫仍不敌，vendor 约 197 TFLOPS 逼近峰值）；而 FlagGems 进程级接管 `aten::mm` 之后，vendor GEMM 从 Python 完全不可达。为此，prefill 大 M 改走 ctypes 直绑 libmcblas 的 `mcblasGemmEx`（cuBLAS 兼容 ABI），并注册为 custom op 保证 compiled 区域安全；decode 细瘦 M 保留自研 Triton nn_db 单布局内核，权重只留一份，不额外占用显存。另修复 `vocab_parallel_embedding` 在 import 时绑死原版 `dispatch_unquantized_gemm` 导致 lm_head 从未进入优化路由的问题，lm_head 由 603 μs 降至 379 μs。

这里的路由不是“看到 vendor 就全切过去”：只有已经生成 nn 副本、且形状满足条件的线性层才进入新路径；小 M 或没有副本时仍走原 `F.linear`。lm_head 则由独立的 `_fl_always_nn` 标志强制走 nn 副本，避免被通用 N 上限误排除。

```
def _metax_unquantized_gemm(layer, x, weight, bias):
    wn = getattr(layer, "_fl_w_nn", None)
```

```
    if wn is not None and bias is None and x.dim() >= 2:
        m = x.numel() // x.shape[-1]
        if (
            getattr(layer, "_fl_always_nn", False)
            or m > _FL_NN_MIN_M
            or weight.numel() == 0
        ):
            x2 = x.reshape(-1, x.shape[-1])
            # Vendor mcBLAS beats the Triton nn kernel 1.35-1.67x at prefill
            # M (sweep_mm_prefill.py); FlagGems' process-wide aten override
            # makes the vendor library unreachable through torch.mm, so the
            # ctypes-bound custom op is the only way to reach it.
            if (
                _mcblas is not None
                and _mcblas.AVAILABLE
                and wn.dtype == torch.bfloat16
```

Listing 3 `linear_nn_repack.py:112--129`：按形状在 vendor、Triton 与 stock 之间路由

ctypes 绑定把行主序的 $C = AB$ 转写成列主序的 $C^T = B^T A^T$，因此不需要改变模型的数学结果；`register_fake` 让编译器在 fake tensor 阶段仍能推导输出形状。

```
def _mm_impl(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    M, K = a.shape
    K2, N = b.shape
    assert K == K2
    out = torch.empty(M, N, device=a.device, dtype=torch.bfloat16)
    st = _lib.mcblasSetStream(
        _handle, ctypes.c_void_p(torch.cuda.current_stream().cuda_stream)
    )
    assert st == 0, f"mcblasSetStream failed: {st}"
    st = _lib.mcblasGemmEx(
        _handle, _MCBLAS_OP_N, _MCBLAS_OP_N,
        N, M, K,
        ctypes.byref(_alpha),
        ctypes.c_void_p(b.data_ptr()), _MACA_R_16BF, N,
        ctypes.c_void_p(a.data_ptr()), _MACA_R_16BF, K,
        ctypes.byref(_beta),
        ctypes.c_void_p(out.data_ptr()), _MACA_R_16BF, N,
        _MCBLAS_COMPUTE_32F, _MCBLAS_GEMM_DEFAULT,
    )
```

Listing 4 `mcblas_mm.py:85--103`：直接调用 mcBLAS，保持原 GEMM 语义

4.4 消除 prefill 每层 DtoH 同步（opt10）

prefill trace 中 GPU 空闲 18.1%，元凶之一是 vendor flash_attn 每层每步执行 `prefill_seq_lens.tolist()`（DtoH 强同步，42 层各一次）。改为纯 GPU cumsum 并缓存到当步

`attn_metadata`，42 层只算一次：4k 提升 10.9%、16k 提升 9.9%，16k TTFT 由 20.8 s 降至 18.0 s。

```
                    attn_metadata,
                    q_descale=layer._q_scale.expand(descale_shape),
                    k_descale=layer._k_scale.expand(descale_shape),
                    v_descale=layer._v_scale.expand(descale_shape),
                )
                return output
            else:
                # ------------------------  Metax Modification -------------------------
                # For handling prefill decode split
                num_decode_tokens = attn_metadata.num_decode_tokens
                if attn_metadata.num_prefills > 0:
                    # fl opt10: compute once per step on GPU (the old code did
                    # tolist()->DtoH sync + HtoD + cumsum PER LAYER = 42 syncs/step)
                    cu_prefix_kv_lens = getattr(
                        attn_metadata, "_fl_cu_prefix_kv_lens", None
                    )
                    if cu_prefix_kv_lens is None:
                        lens = attn_metadata.prefill_seq_lens
                        cu_prefix_kv_lens = torch.cat(
                            [lens.new_zeros(1), lens]
                        ).cumsum(dim=0, dtype=torch.int32)
```

Listing 5 `flash_attn.py:790--810`：把每层一次的 DtoH + cumsum 改成当步缓存

这段代码只改变 metadata 的生产方式，不改变请求组成、调度顺序或 `vllm serve` 参数；缓存对象仍然属于当前 `attn_metadata`，不会跨请求复用旧长度。

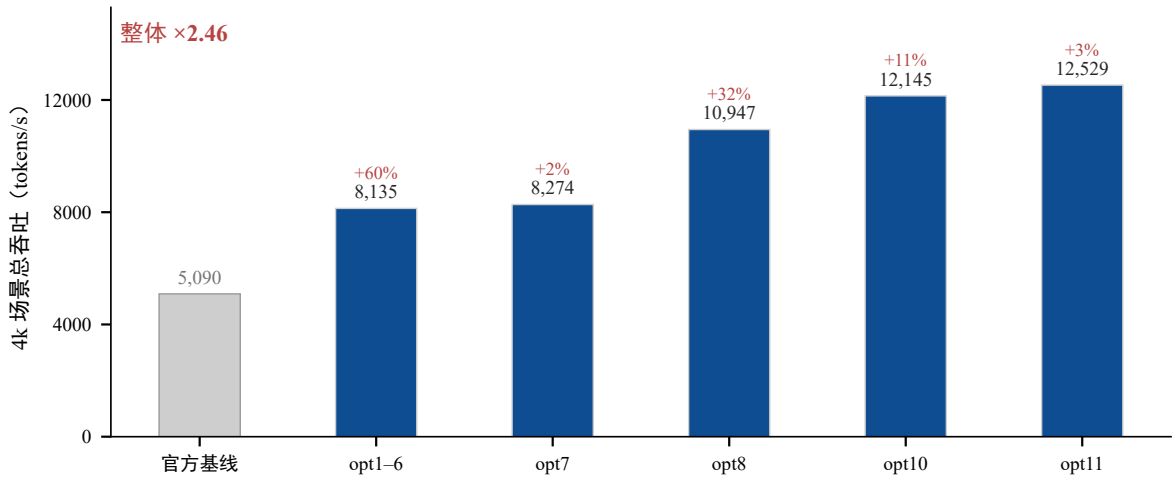


图 8 C500 4k 场景优化阶梯（total tokens/s）

5 天数 BI-V150：vendor 接入、发射配置与自研 decode 内核

5.1 prefill attention 接入 vendor ixAttnBkd（A1 / A6）

iluvatar 后端缺少 flash_attn 包时，attention 落到 vLLM 通用 Triton 参考实现，16k prefill 每层每 2048-chunk 需 36.8 ms，占 GPU busy 91.8%。接入 vendor ixAttnBkd 内核并确认其 causal 语义为 bottom-right（cmt=2）与 vLLM 的 chunk prefill 对齐后，内核级实测在所有 kv 长度上均快 12–20 倍。配套地，builder 一次性完成每步 decode/prefill 划分，避免每层两次 D2H；KV 用自研 Triton `_dual_gather_kernel` 一次完成 K+V 分页收集（bit-exact，GPU 106→62 μs、host 68→37 μs），varlen 分支用 out= 直写消除 copy-out，每请求发射数由 4 降为 2。

```
    def _dual_gather_kernel(KVC, PAGES, KP, VP,
                            E: _tl.constexpr, BLOCK: _tl.constexpr):
        # kv_cache [blocks, 2, bs, hkv, d] flat: page p's K chunk at
        # p*2*E, V chunk right after.  One program per page copies both.
        pid = _tl.program_id(0).to(_tl.int64)
        page = _tl.load(PAGES + pid)
        offs = _tl.arange(0, BLOCK)
        src = KVC + page * (2 * E)
        dst = pid * E
        for i in range(0, E, BLOCK):
            _tl.store(KP + dst + i + offs, _tl.load(src + i + offs))
            _tl.store(VP + dst + i + offs, _tl.load(src + E + i + offs))
except Exception:
```

Listing 6 `vendor_flash_attn.py:46--58`：同一 Triton program 收集一页 K 与 V

这里的 `E=bs*hkv*d` 正好是一页 K 的元素数，V 的源地址就是 K 之后的连续区域；所以它不是“两次 index_select 的包装”，而是利用页内布局把两次遍历合并成一次，且已用 bit-exact 探针验证。

5.2 decode unified-attention 发射配置重构（A7）

stock Triton 默认 `num_warps=4, num_stages=3` 是面向 NVIDIA 数据中心的甜点配置，在 BI-V150 上系统性失配：软件流水线在每个 warp 档上都是负收益，warps 越少越好。离线扫描 3D 主内核与 reduce_segments 全链路后，仅对 3D flash-decoding 路径固定为 w1 s1（TILE=16 / SEG=16 经扫描本已最优），输出与 stock bit-exact（max_diff=0.0）。这一项同时说明，配置维度是独立于算法维度的优化空间。

```
    class _PinnedLaunch:
        def __getitem__(self, grid):
            fn = kern[grid]

            def _call(**kw):
```

```
                if kw.get("IS_3D"):
                    kw["num_warps"] = _NUM_WARPS
                    kw["num_stages"] = _NUM_STAGES
                return fn(**kw)

            return _call

    _tua.kernel_unified_attention = _PinnedLaunch()
```

Listing 7 decode_attn_pin.py:35--47：只包住 3D 内核的发射配置，2D 路径不变

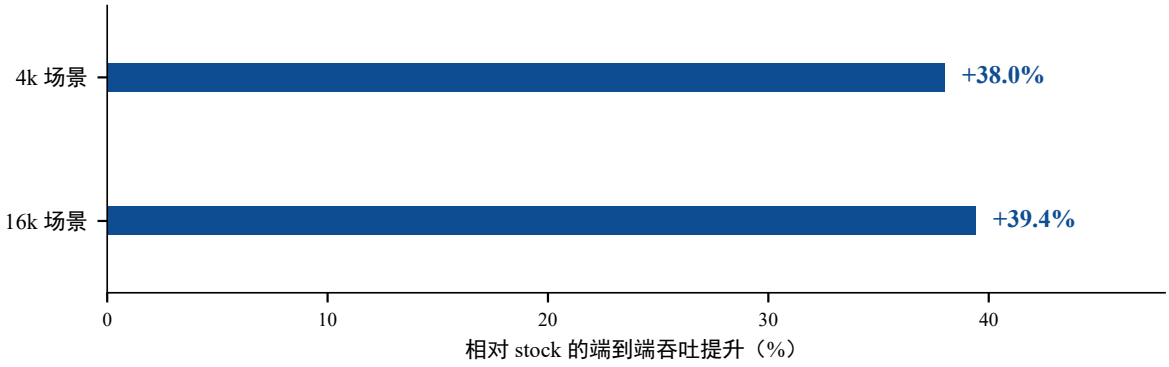


图 9 A7 发射配置重构带来的端到端吞吐提升

5.3 自研 paged decode attention 内核（A20）

在 A7 之后，decode attention 仍是整步中最大且唯一未压到带宽下限的一项。把 stock kernel 逐段拆开（同一 harness，B=64 / kv=4096 / 16q-2kv / d=128 / block=16 / bf16）可见：loads+QK 已跑到 509 GB/s，softmax 几乎免费，而 PV 的 `tl.dot` 独占 0.36 ms——它必须把 p 从 MMA 累加器布局转换为 A 操作数布局，每个 KV block 都要过一次共享内存往返。瓶颈是布局搬运，不是带宽。

据此重写内核（算子位于 `FlagGems/fused/paged_decode_attention.py`，插件侧为薄封装与派发）：一个 CTA = 1 warp =一个（seq, kv_head, 段），使布局转换留在 warp 内；BLOCK_N=16，p tile 不超过一个 MMA 形状；base-2 在线 softmax，scale 预折 $\log_2(e)$ 走 MUFU；主循环不加 mask，仅 ragged 尾部单独一个带 mask 的 block；段数固定、段长由 device 端 seq_lens 现算（CUDA-graph 安全），段缓存 slab 在 `apply()` 即捕获前一次性预分配；段边界按整页对齐，进一步消去逐 token 除模与向量页 gather。混合步中的 decode 行也通过 `try_fast_decode` 进入同一内核。

下面的主循环展示了三个直接针对瓶颈的选择：页边界对齐后 `page` 只需标量加载；`exp2` 代替 `exp`；每段只写自己的 `SEG_ACC/SEG_MAX/SEG_SUM`，最后由 combine 按 $\exp 2(m_i - m)$ 加权归并。



主循环无 mask；仅 ragged 尾块 mask；段长 device 端计算，slab 在图捕获前预分配

图 10 A20 paged decode attention 的分段执行与归并流程

```
    for t0 in range(start, end, BS):
        page = tl.load(BT + pid_s * stride_btb + (t0 // BS) * stride_btp)
        if t0 + BS <= kvlen:
            kt = tl.load(K + page * stride_kb + pos[None, :] * stride_ks
                         + pid_h * stride_kh + offs_d[:, None] * stride_kd)
            vt = tl.load(V + page * stride_vb + pos[:, None] * stride_vs
                         + pid_h * stride_vh + offs_d[None, :] * stride_vd)
            s = tl.dot(qv, kt) * scale
        else:
            mask_t = t0 + pos < kvlen
            kt = tl.load(K + page * stride_kb + pos[None, :] * stride_ks
                         + pid_h * stride_kh + offs_d[:, None] * stride_kd,
                         mask=mask_t[None, :], other=0.0)
            vt = tl.load(V + page * stride_vb + pos[:, None] * stride_vs
                         + pid_h * stride_vh + offs_d[None, :] * stride_vd,
                         mask=mask_t[:, None], other=0.0)
            s = tl.where(mask_t[None, :], tl.dot(qv, kt) * scale,
                         float("-inf"))
        m_new = tl.maximum(m_i, tl.max(s, 1))
        alpha = tl.exp2(m_i - m_new)
        p = tl.exp2(s - m_new[:, None])
        l_i = l_i * alpha + tl.sum(p, 1)
        acc = acc * alpha[:, None] + tl.dot(p.to(vt.dtype), vt)
        m_i = m_new
```

Listing 8 `paged_decode_attention.py:169--193`：页对齐主循环与 base-2 在线 softmax

```
    pid_s = tl.program_id(0)
    pid_h = tl.program_id(1)
    offs_g = tl.arange(0, GQA)
    offs_d = tl.arange(0, D)
    offs_n = tl.arange(0, NSEG)

    base = (pid_s * HKV + pid_h) * NSEG
    mx = tl.load(SEG_MAX + (base + offs_n)[None, :] * GQA + offs_g[:, None])
    sm = tl.load(SEG_SUM + (base + offs_n)[None, :] * GQA + offs_g[:, None])
    m = tl.max(mx, 1)
    w = tl.exp2(mx - m[:, None])
    denom = tl.sum(w * sm, 1)
    accs = tl.load(SEG_ACC + ((base + offs_n)[None, :, None] * GQA + offs_g[:, None, None]) * D
                   + offs_d[None, None, :])
```

```
    out = tl.sum(accs * w[:, :, None], 1) / denom[:, None]
    tl.store(OUT + pid_s * stride_ob + (pid_h * GQA + offs_g)[:, None] * stride_oh
             + offs_d[None, :] * stride_od, out.to(OUT.dtype.element_ty))
```

Listing 9 paged_decode_attention.py:207--223：分段结果的在线归并

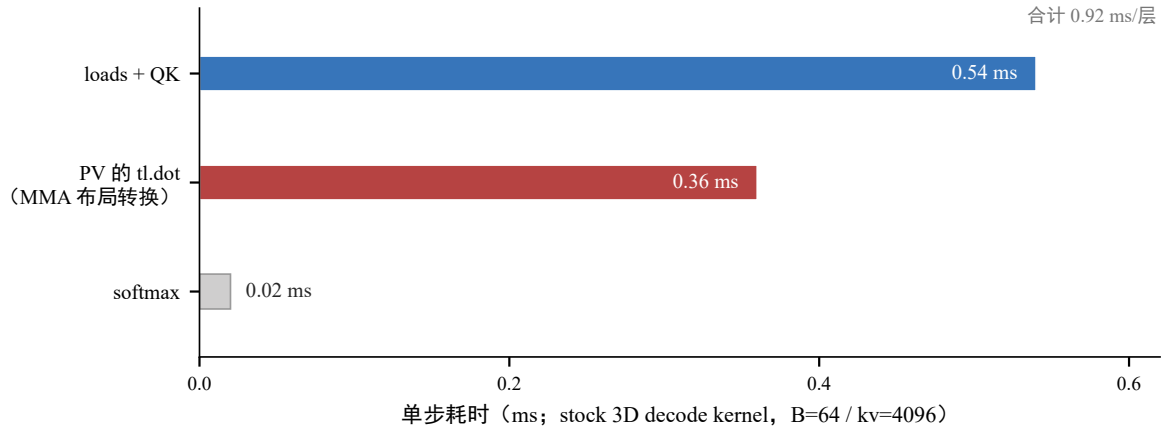


图 11 stock 3D decode 内核的耗时分解：瓶颈在 PV 的 MMA 布局转换

内核级结果为 1.04 → 0.72 ms/层（ragged 长度下 0.90 → 0.66 ms），输出与 stock 的 max|diff| 为 2.4×10^{-4}（ragged 为 9.8×10^{-4}）。同日交错 A/B：4k 由 2239.43 提升至 2606.56 tok/s（+16.4%），16k 由 2956.59 提升至 3541.34 tok/s（+19.8%）。精度 MATH-500 Level 3 四次复核为 96.2% / 97.1% / 98.1% / 97.1%，均达标。

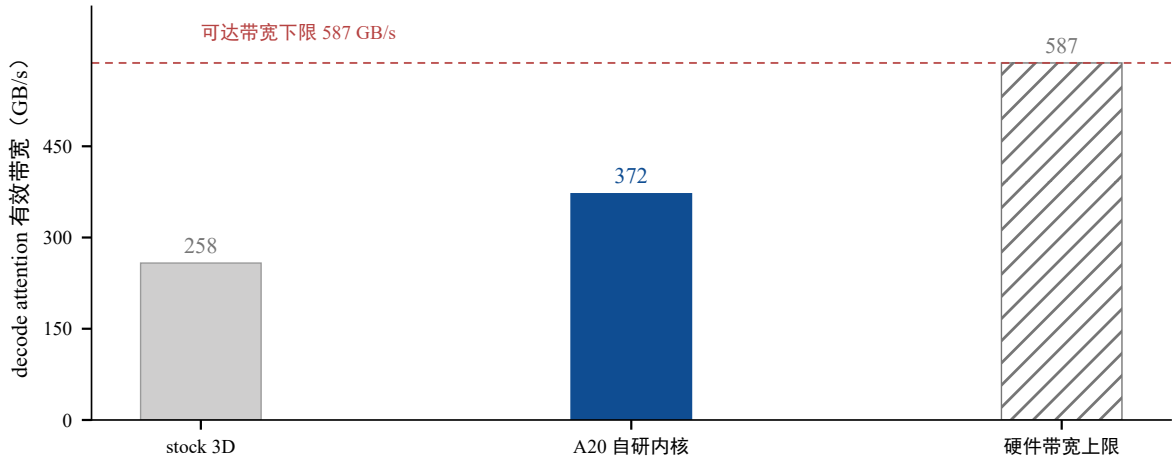


图 12 decode attention 有效带宽对比（kv=4096，B=64）

5.4 可复现性与可审计性

内核每个（seq, kv_head, segment）只依赖自己的 q / KV 页与 seq_lens，段长只由该序列长度决定，因此同一序列的输出与批组成无关。FlagGems 侧配有 tests/test_paged_decode_

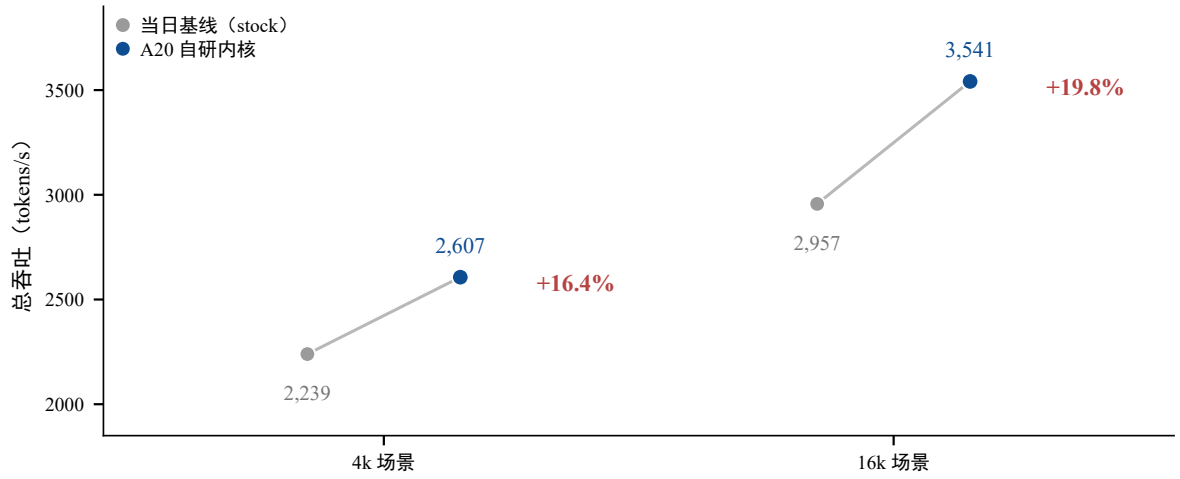


图 13 天数 BI-V150：A20 同日交错 A/B 结果

`attention.py`：8 个精度用例对照 fp32 参考，外加一个批量不变性用例（单条 / 两条 / 半批 / 全批 / 逆序全批逐位相等，并重复同组成验证确定性），BI-V150 上 9 passed。混合步的 decode 切片每 4000 次调用输出一行 used/skipped 计数，16k 两次实测均为 used=44000、skipped=0，即 100% 命中自研内核，不存在隐式退回 stock 的情况。

```
def try_fast_decode(impl, query, kv_cache, md, out):
    """Entry point for the mixed-step (prefill+decode) path.

    Returns True when the fast kernel computed `out` for the decode slice.
    A periodic INFO line reports the engagement ratio so a run can be
    audited from its serve log alone.
    """
    if _DISABLED or md is None:
        return False
    if not _eligible(impl, query, kv_cache, md, None):
        _slice["skip"] += 1
        _slice_report()
        return False
    _slice["used"] += 1
    _slice_report()
    try:
        return _fast_forward(impl, query, kv_cache, md, out) is not None
    except Exception as exc:
        logger.warning("decode_attn_fast: slice fallback (%s)", exc)
        return False
```

Listing 10 `decode_attn_fast.py:186--205`：混合步 decode 切片的资格检查、计数与异常回退

因此“自研内核命中”和“stock 回退存在”是可同时审计的：满足条件才进入快路径，任何不支持的特征或异常都回到原实现；日志里的 used/skipped 不是装饰性计数，而是验证优化是否真正落到官方混合批上的依据。

6 正确性与精度证据

赛题精度口径是 MATH-500 Level 3 的题目级二分类（答对 / 答错），但当前 evalscope 汇总文件只保存 105 题的总分，没有逐题 prediction、gold 和 correctness，因此本报告不虚构混淆矩阵。精度证据分为四层：内核数值等价、批量与调度不变性、采样分布等价、端到端官方评测。这样既回答“模型是否仍然答对”，也回答“优化是否真的按预期执行”。

表 4 正确性验证矩阵：从单个 kernel 到完整服务

验证层	对象与方法	实测结论
bit-exact	A6 双 gather、A7 发射配置	A6 与两次 `index_select` 逐位一致；A7 与 stock 的 `max_diff`=0.0
数值误差	A20 vs stock，含 ragged 形状	4k 最大 2.4×10^{-4}，ragged 最大 9.8×10^{-4}
批量不变性	单条 / 两条 / 半批 / 全批 / 逆序全批，对照 fp32	9 个用例通过；同组成重复执行也确定
采样等价	greedy、top-p、Gumbel 与 TV 距离	greedy 与 `torch.argmax` 完全一致；保留集 IoU ≥ 0.999；TV 与参考实现同档
端到端精度	evalscope MATH-500 Level 3（105 题）	C500 97.14%–98.1%；BI-V150 96.2%–98.1%，均超过 95%
真实路径命中	混合步 decode 切片 used/skipped	两次 16k 均为 used=44000、skipped=0，无隐式回退
交付可运行	全新 clone + 独立 venv + 官方启动	C500 启动、`/v1/models` 与实际 chat 请求均通过

如果后续需要严格的 2×2 混淆矩阵，应在 evalscope 阶段额外保存每题的 correctness，再按“基线答对 / 答错”与“本提交答对 / 答错”交叉计数；它不能由 97.14% 或 98.1% 这样的汇总准确率反推出来。本报告选择保留这个边界，不把不存在的逐题数据包装成分析结果。

7 主要创新点

（1）跨双平台的 shape-aware dispatch 统一框架。同一套“离线全空间扫描＋派发层按 shape 选择实现/配置”的方法论落在两种国产芯片上，覆盖 GEMM、attention、采样三类算子，且有赛题组明确的合规口径；本报告同时给出该框架的适用边界（见第 8 节 A26）。

（2）免排序 top-p 采样。1024-bin 质量直方图 zoom 替代全词表排序，保留集质量恒不小于 $p \cdot Z$，是有下界保证的确定性构造而非近似猜阈值；双平台复用，采样链约 40 倍提速。

（3）发射数削减范式。dual-gather 利用 kv_cache 页内 K/V 连续布局一次完成收集，配合 varlen out= 直写，把 prefill attention 的 host 开销从每请求 4 次发射降到 2 次，全部经 bit-exact 探针验证。

（4）配置维度的系统性诊断。指出 NVIDIA 默认启发式（w4s3）与 libtuner 在 event-timing 降级模式下于国产芯片上的失配根因，仅这一项即带来 4k/16k 各约 40% 的端到端收益。

（5）自研 decode attention 内核。不是泛泛地“重写 flash attention”，而是先把 stock kernel 逐段拆开，定位到 PV 的 MMA 累加器布局转换这一具体瓶颈，再用单 warp、小 tile、base-2

softmax、尾部 mask 与页对齐分段逐项消除，使 decode attention 带宽由 258 GB/s 提升到 372 GB/s；并用“跳过 attention 的探针”量出该方向的硬件下限（19.3 ms/步）。

（6）零精度妥协与可复核闭环。所有性能改动要么 bit-exact，要么统计等价且通过 MATH-500 Level 3 实测；每个候选都有当日交错 A/B、原始 CSV、机器状态与回退开关，未通过 3% 准入或精度门槛的一律不进入提交。

8 阴性结果与探索覆盖度

下列方案均经过实测后被否决，未进入提交代码。保留它们是为了说明探索的边界，也便于评审复核“为什么停在这里”。

表 5 主要被否决方案及其证据

方案	隔离/理论收益	端到端结果与结论
A2：自研 Triton flash-decoding	与 stock 打平（默认配置）	转投配置维度，由 A7 获益
opt9：C500 attention 内核选择	全空间扫描	已达 vendor 上限
A8a：decode host 壳子	—	收益 <1%，混合步占比低且 GPU-bound
A8b：lm_head GEMM 48 组配置	—	aten vendor 已贴带宽墙
A9：n-gram 投机采样探针	draft 质量看似可用	75% 请求零命中，期望收益 <10%，否决
A20-int8 KV	期望降低 KV 带宽	只加载就 1.06 ms/层 vs bf16 0.72，1 字节 gather 吞吐仅 2 字节的 1/3
A20-paged prefill	期望省反分页 gather	gather 仅占 prefill GPU 时间约 1.5%，且 API 不兼容
A21：IR 级 RMSNorm 融合	隔离 2.5–7.2×，逐位一致	引擎内 −5.9%、TTFT 翻倍；IR 被 inline 后失去相邻融合
A22 / A27：W8 权重量化	修正实现后隔离 1.82×	引擎内 −19%（4k 2080 vs 2571），TTFT 恶化 4.4 倍
A23 / A24：aten::index 退化	同形状 index_select 快 13000 倍	属隐式同步点；修复后投机混合步仍饥饿，不采用
A25：_build_split 的 81 ms/步	该项 81.4 → 0.3 ms	官方 4k 仅 +0.85%，属等待而非浪费
A26：shape-aware NSEG 分派	小 batch 有 3–7.4%	官方 c64 下 NSEG=32 已最优、带宽 88%，收益上限 1.4%
C500 num_splits 调优	微基准 3 倍差异	冷缓存实测 10894 / 10953 vs 基线 11104，无收益
图内 CUDA event 计时	—	invalid resource handle，本栈不支持

其中 A22 / A27 值得单独说明：最初以 0.77× 否决 W8，是因为量化被拆成 abs / max / float / quant 四个 launch，且基线误用了不含 SwiGLU 的裸 GEMM。修正为单核 per-token 量化加融合 dequant-silu 后，隔离测量确为 1.82×；但接入真实引擎后端到端 −19%。这说明在本卡上，微基准只能用于筛掉明显更差的设计，不能用于预测端到端收益——同一层连调百次的“热权重”与 42 层轮转的“冷权重”是两种工况。

表 6 赛题约束与本提交做法

约束	本提交做法
`vllm serve` 参数与基线一致	两张卡均按官网原命令启动，未增删任何参数；C500 不额外添加 `--compilation-config`
禁止 benchmark 脚本调优	一律使用官方脚本与官方聚合口径，未修改脚本
禁止直接 merge 上游分支	基线为 flagos-2026-s2 与 FlagGems v5.3.5，全部改动以补丁形式叠加
算子优化须有开发量	新增 Triton 内核（paged decode attention、采样链、dual-gather）与路由/同步改造，均可独立 A/B
禁止改超参开启量化、投机采样	W8 权重量化与 n-gram 投机均在评估后未采用，未进入提交
精度 ≥ 0.95	MATH-500 Level 3：C500 97.14%–98.1%，BI-V150 96.2%–98.1%
TTFT 不回退	各场景 TTFT 均优于基线；16k C500 由 27197 ms 降至 17902 ms

9 合规对照与复现指南

9.1 合规对照

9.2 复现步骤

先取得两个集成分支（保留完整历史），再分别以可编辑模式安装：

```
git clone --single-branch --branch integration/vllm-plugin-fl \
  https://github.com/xy200303/vllm-plugin-FL.git vllm-plugin-FL
git clone --single-branch --branch integration/flaggems \
  https://github.com/xy200303/FlagGems.git FlagGems

cd vllm-plugin-FL && pip install -e . --no-build-isolation
cd ../FlagGems   && pip install -e . --no-build-isolation
```

沐曦 C500 还需设置以下环境变量（天数无需）：

```
export PATH=/opt/conda/bin:$PATH
export MACA_PATH=/opt/maca
export MACA_HOME=/opt/maca
export LD_LIBRARY_PATH=/opt/maca/lib:/opt/maca/lib64
export VLLM_PLUGINS=fl
```

启动命令与官网完全一致，天数在官方命令后追加编译配置，C500 不追加：

```
# 天数 BI-V150
vllm serve /workspace/MiniCPM5-2B --port 9031 --served-model-name minicpm \
  --gpu-memory-utilization 0.85 --max-model-len 131072 \
  --compilation-config '{"cudagraph_mode": "FULL_DECODE_ONLY"}'

# 沐曦 C500（不添加 --compilation-config）
vllm serve /workspace/MiniCPM5-2B --port 9031 --served-model-name minicpm \
  --gpu-memory-utilization 0.85 --max-model-len 131072
```

性能评测执行官方脚本并传入两个官方用例：

```
python3 vllm-plugin-FL/benchmarks/benchmark_throughput_serve.py \
  --model /workspace/MiniCPM5-2B --served-model-name minicpm --port 9031 \
  --test-cases '[[4096,1024,64,256],[16384,1024,64,128]]'
```

精度评测执行官网给出的 evalscope 命令，结果见 `reports/minicpm/math_500.json` 的 score 字段。

9.3 复现注意事项

（1）防止 prefix cache 污染：benchmark 使用 seed=0，同一 serve 上连跑两次相同 input_len 的负载会产生完全相同的 prompt，第二次会大量命中 prefix cache（本队日志出现过 35.9% 命中率），使 prefill 近乎免费、总吞吐虚高 30% 以上。每个数据点都应在新启动的 serve 上获取，A/B 两侧均从冷缓存开始。

（2）C500 需要 ≥128 GB CPU 内存：在模力方舟 64 GB 内存规格实例上，纯镜像原版 `vllm serve` 也会在 inductor 编译阶段被 cgroup OOM killer 杀掉（匿名内存峰值 63.4 GB，被杀时 GPU 显存仅 858 MiB，与显存、负载、代码改动均无关）。128 GB 规格实例全程稳定。

（3）首次启动耗时：C500 需执行 GEMM autotune 预热，实测两台实例分别为 367 秒与 973 秒；第二次启动（triton cache 已存在）约 1–3 分钟。此为正常现象，不是卡死；如确需跳过可设 `VLLM_FL_MM_WARMUP=0`，但 4k 吞吐约 −6%、TTFT 变差。

（4）实例间差异：两台 C500 实例的 4k 吞吐相差 4%、TTFT 相差 66%，16k 几乎一致。报告因此以区间给数，不以单台最优值充当普遍成绩。

10 结论与建议

10.1 结论

本报告在官方指定的 FlagOS 栈上，完成了 MiniCPM5-2B 在天数 BI-V150 与沐曦曦云 C500 两款国产芯片上的推理吞吐优化。在不修改 `vllm serve` 参数、不更换模型、不降低精度的约束下，C500 平台 4k 场景达到官方基线的 2.33–2.44 倍、16k 场景 1.58–1.59 倍；天数平台 4k 由 2028.01 提升至 2606.56 tok/s（+28.5%）、16k 由 915.15 提升至 3541.34 tok/s（+287%）；MATH-500 Level 3 精度保持在 96.2%–98.1%，高于 95% 的门槛，TTFT 在各场景均未回退。

从方法论上看，本工作验证了一条可复用的路径：先用微基准把时间归因到具体内核或同步点，再用最小改动验证，最后以官方口径做当日交错 A/B。这条路径既解释了为什么某些看似有利的方案（W8 量化、投机采样）在端到端上不可行，也解释了为什么某些微小改动（发射配置、双 gather）能带来数十个百分点的收益。

10.2 局限与后续工作

其一，本文的绝对值受开发实例状态影响较大，跨日不可比，因此天数侧一律以同日交错 A/B 的相对收益表述；其二，C500 侧在两台实例上的 4k 吞吐存在约 4% 的差异，报告因此给区间而非单点；其三，decode attention 在内核层面已接近该卡的访存上限（C500 实测 1466 GB/s，天数 decode kernel 372 GB/s 对应其可达负载带宽的约 70%），进一步提效需要脱离 Triton 或引入更低比特的 KV 表示，而后者在本文的实验中已因 gather 吞吐退化被否决。

后续工作可从三个方向展开：一是把 shape-aware 分派扩展到 prefill 侧的分块策略；二是探索编译器层面的算子融合（本文的 IR 级 RMSNorm 尝试因 inductor 失去相邻融合而失败，但换一种插入位置可能成立）；三是在更多国产芯片上复现同一套测量闭环，检验其可迁移性。

另外，当前上游有两项值得独立验证的参考实现：common attention metadata graph（#442）与图捕获前 prewarm CachedOps（#552）。它们分别对应本文的 metadata 同步与算子路由预热问题，后续将各自新建实验分支，先验证设备兼容性，再做官方 A/B；本报告的性能数字不包含这两项工作，也不直接 merge 上游主分支。

参考文献

[1] 中华人民共和国国家质量监督检验检疫总局, 中国国家标准化管理委员会. GB/T 7713.3-2014 科技报告编写规则 [S]. 北京: 中国标准出版社, 2014.

[2] KWON W, LI Z, ZHUANG S, et al. Efficient memory management for large language model serving with PagedAttention[C]//Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP). 2023: 611-626.

[3] DAO T, FU D Y, ERMON S, et al. FlashAttention: fast and memory-efficient exact attention with IO-awareness[C]//Advances in Neural Information Processing Systems (NeurIPS). 2022.

[4] MADDISON C J, MNIH A, TEH Y W. The concrete distribution: a continuous relaxation of discrete random variables[C]//International Conference on Learning Representations (ICLR). 2017.

[5] FlagOS Community. vllm-plugin-FL, branch flagos-2026-s2[EB/OL]. https://github.com/flagos-ai/vllm-plugin-FL.

[6] FlagOS Community. FlagGems, tag v5.3.5[EB/OL]. https://github.com/flagos-ai/FlagGems.

[7] OpenBMB. MiniCPM5-2B[EB/OL]. https://modelscope.cn/models/OpenBMB/MiniCPM5-2B.

A 代码与实验记录索引

代码分支：插件为 xy200303/vllm-plugin-FL 的 integration/vllm-plugin-fl（C500 opt1 – opt11、天数 A1 – A7 与 A20，至 0207b7e1）；算子为 xy200303/FlagGems 的 integration/flaggems（采样融合内核、GEMM、paged_decode_attention，至 dd574fdd）。两者均为对应上游项目的正式 fork；历史开发线归档于 archive/*，官方基线保留于 baseline/*。报告与实验记录另存于 xy200303/minicpm-flagserve 的 main。

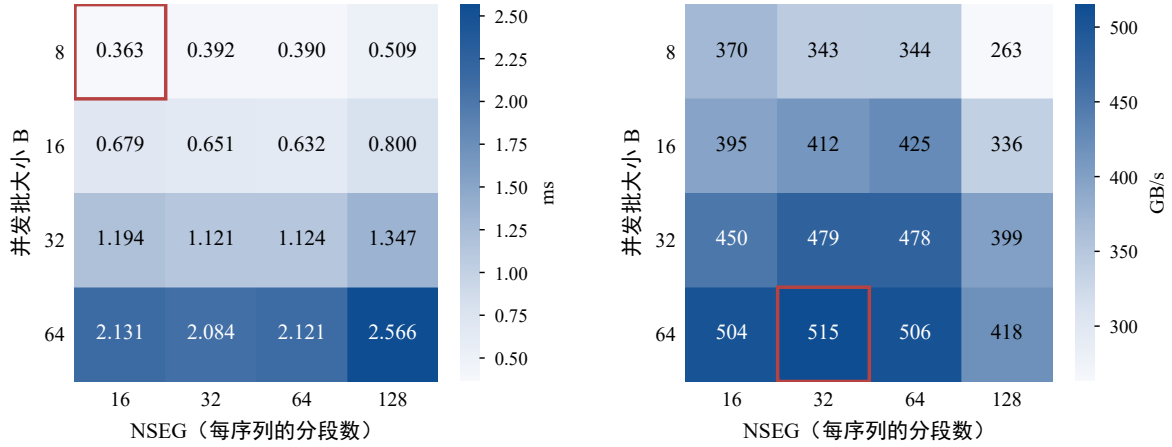
表 7 关键代码文件与报告章节的对应关系

文件（相对各自源码根目录）	作用与对应章节
vllm_fl/dispatch/backends/vendor/metax/patches/sampler_tail_fusion.py	C500 采样链接入，§4.2
vllm_fl/dispatch/backends/vendor/metax/patches/linear_nn_repack.py	nn 权重复本与形状路由，§4.3
vllm_fl/dispatch/backends/vendor/metax/patches/mcblas_mm.py	ctypes 直调 vendor mcBLAS，§4.3
vllm_fl/dispatch/backends/vendor/metax/impl/attention/flash_attn.py	prefill metadata 的当步 cumsum 缓存，§4.4
vllm_fl/dispatch/backends/vendor/iluvatar/impl/vendor_flash_attn.py	vendor prefill、混合步拆分、双 gather，§5.1
vllm_fl/dispatch/backends/vendor/iluvatar/impl/decode_attn_pin.py	3D decode 发射配置，§5.2
vllm_fl/dispatch/backends/vendor/iluvatar/impl/decode_attn_fast.py	A20 资格检查、slab、混合步回退，§5.3–5.4
src/flag_gems/fused/top_p_sample.py	直方图 top-p 与 Gumbel-Max，§4.2
src/flag_gems/fused/paged_decode_attention.py	A20 分段内核与 combine，§5.3
tests/test_paged_decode_attention.py	精度与批量不变性测试，§5.4、§6

实验记录：docs/baselines/metax-c500/（C500 基线与 opt1 – opt11 逐项记录、2026-10-23/24 复核与验证）、docs/baselines/iluvatar-biv150/（天数 A1 – A9 记录）、docs/experiments/（A10 – A27 的探针、台账 tracking.md、runs/ 原始数据与证据）。所有候选方案的配置、serve 日志、benchmark CSV、机器状态与结论均可按目录复核。

B 内核形状扫描与测量可靠性

本附录给出两类支撑材料：其一为 paged decode attention 内核在 NSEG × 并发度上的实测分布（来源：docs/experiments/a26/a26_sweep.log，20 次重复取均值）；其二为官方 benchmark 连续 4 次 run 的吞吐波动（来源：docs/baselines/metax-c500/raw_runs_baseline.csv）。前者说明当前配置为何是甜点，后者说明测量口径的离散程度。



(a) 单层耗时（ms/层） (b) 有效带宽（GB/s）

图 14 NSEG × 并发度的实测分布（kv=16384）：红框为各自最优格

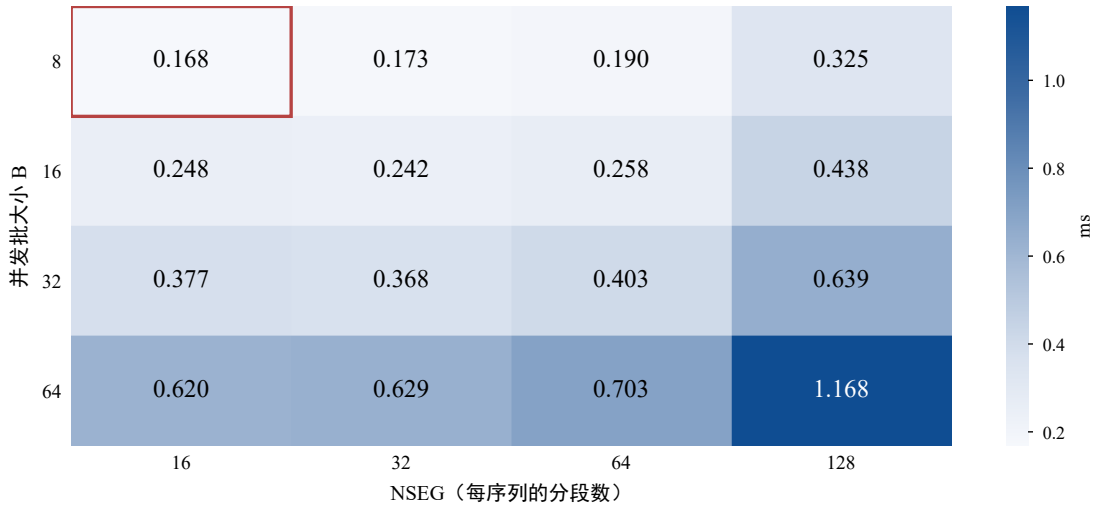


图 15 4k 场景下 NSEG × 并发度的单层耗时分布（kv=4096，ms/层）

三张热力图显示：在官方 c64 形状下，NSEG=32 恰为带宽最优点（515 GB/s，约为该卡理论带宽的 88%），继续增大 NSEG 反而因归并开销退化；小 batch 虽可通过调整 NSEG 获得 3%–7.4% 的收益，但官方两个用例均为 c64，因此未实现 shape-aware 分派（对应第 8 节 A26）。

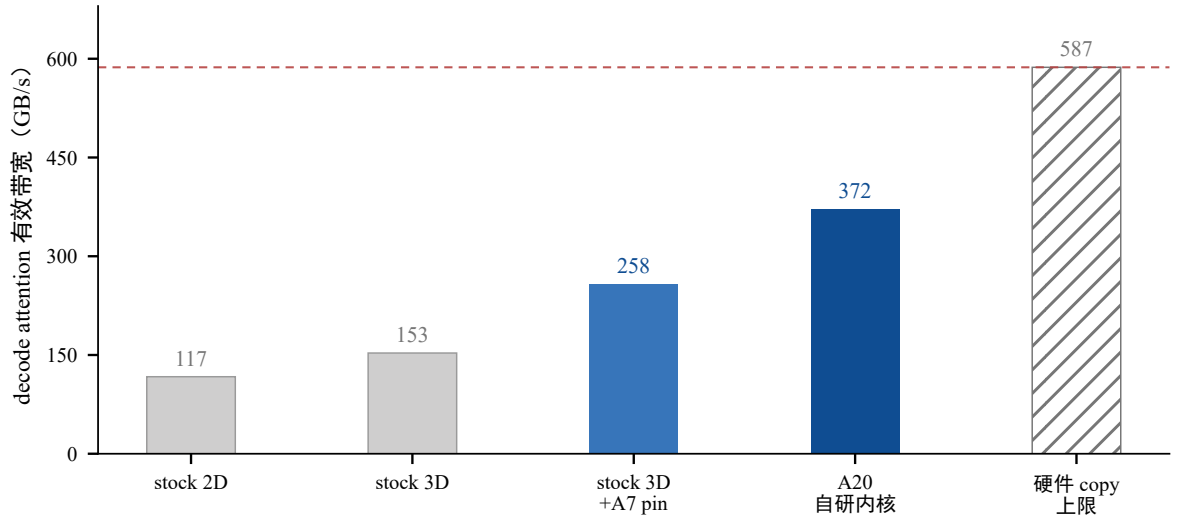


图 16 decode attention 从 stock 到自研内核的带宽演进

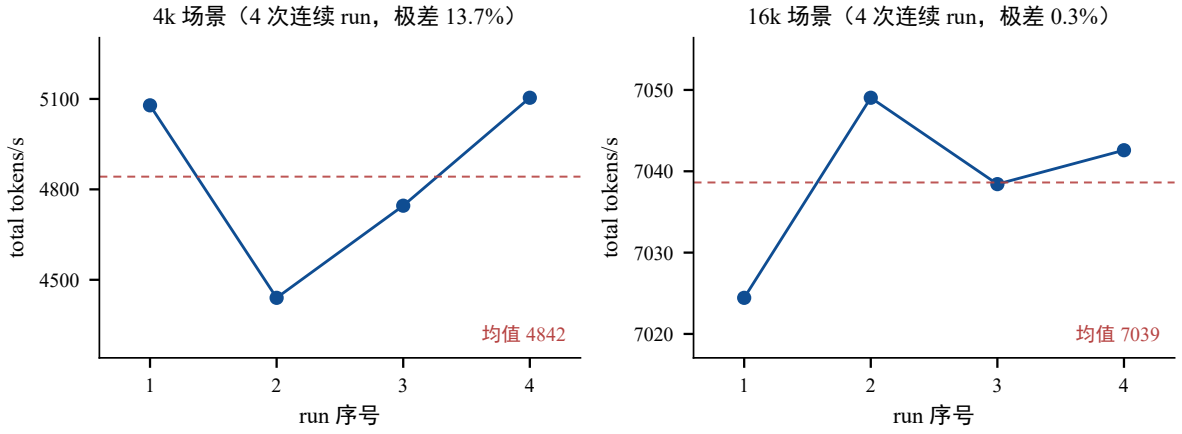


图 17 官方 benchmark 连续 4 次 run 的吞吐波动：4k 需取均值，16k 极稳

由此确定报告口径：4k 场景取 4 次 run 的均值，16k 场景因极差仅 0.3% 采用单次测量；所有优化项的收益一律以同日交错 A/B 给出，不用跨日绝对值，从而把时段漂移从结论中剔除。